Cloud Storage

Облачное хранилище файлов

Описание функциональности, архитектуры и принципов работы проекта

Технологии: Go / gRPC / grpc-gateway · nginx · MongoDB · S3 / VersityGW · Docker Compose · Prometheus · Grafana

Проект представляет собой веб-сервис для хранения личных файлов с разграничением доступа, пользовательскими квотами и публикацией ссылок. Система включает отдельные сервисы хранения данных, обработки запросов и наблюдаемости.

Содержание	Страница
Назначение и возможности	2
Архитектура и модель данных	3-4
Авторизация, загрузка и жизненный цикл файла	5-7
Мониторинг и оповещения	8-9
Развёртывание и API	10-11
Демонстрация, ограничения и карта исходного кода	12-14
gRPC и разделение слоёв приложения	15

Документ подготовлен по исходному коду и конфигурации репозитория на 09.11.2026. Описывает реализованное поведение; Нагрузочные испытания и кластерная отказоустойчивость не заявляются.

РАЗДЕЛ 01

Назначение и возможности

Цель проекта - реализовать собственный сервис облачного хранения и показать взаимодействие контейнерных компонентов: веб-клиента, API, базы метаданных, объектного хранилища и средств мониторинга.

Пользователь работает с файлами через браузер. API проверяет права и ограничения, сохраняет содержимое через S3-совместимый интерфейс, а сведения о файле - в MongoDB. Доступ к файлам возможен из любого браузера, имеющего сетевой доступ к развёрнутому сервису.

Возможность	Как проявляется для пользователя
Регистрация и вход	Учётная запись по email и паролю; сессия на основе JWT.
Загрузка	Выбор одного или нескольких файлов, drag&drop, индикатор прогресса. Несколько выбранных файлов отправляются последовательно.
Каталог и поиск	Имя, размер и дата загрузки; поиск по подстроке имени без учёта регистра.
Операции с файлом	Скачивание, открытие поддерживаемого типа в браузере, переименование и удаление.
Публичный доступ	Создание ссылки без обязательного входа получателя и последующее закрытие доступа.
Квоты	Показ занятого места; ограничение общего объёма пользователя и размера одного файла.
Оповещения	Просмотр активных и решённых инфраструктурных проблем.

Связь с дисциплиной «Облачные сервисы»

Реализация демонстрирует сетевой доступ к сервису, многопользовательскую изоляцию, объектную модель хранения, учёт потребления ресурсов, контейнеризацию и централизованный мониторинг. Локальный запуск через Compose моделирует инфраструктуру частного облачного сервиса.

Текущий стенд работает на одном Docker-хосте. Автомасштабирование, кластерная отказоустойчивость и географическая репликация в конфигурации не реализованы.

РАЗДЕЛ 02

Архитектура системы

Основной путь запросов. Мониторинг показан отдельно в разделе 07.

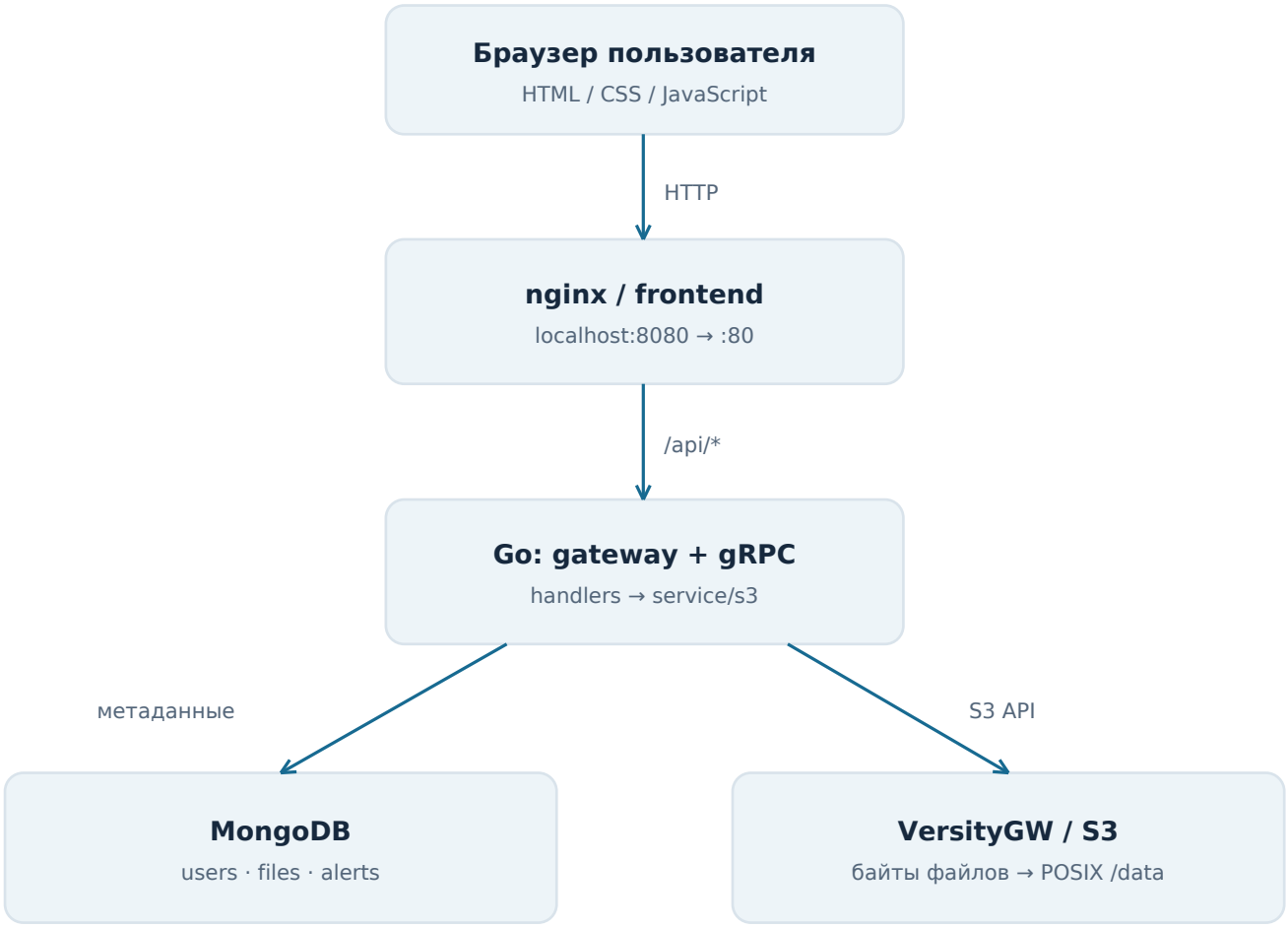


Рисунок 1. Основные компоненты и направления взаимодействия.

Frontend - статические HTML, CSS и JavaScript без клиентского фреймворка. nginx раздаёт интерфейс и проксирует **/api/** в Go backend. Путь **/swagger/** направляется в отдельный Swagger UI.

Backend - единое Go-приложение: HTTP gateway :8000, gRPC :9000 и отдельный бизнес-сервис. Контракт задаётся в protobuf, HTTP-маршруты генерируются по аннотациям. Браузер получает файлы через backend, а не напрямую из бакета.

MongoDB отвечает за пользователей и каталог. **VersityGW** предоставляет S3 API поверх POSIX-каталога: реальные байты хранятся в Docker volume **s3-data**. Внутри сети контейнеры обращаются друг к другу по именам сервисов.

В Compose определены 8 сервисов: mongo, versitygw, statsd-exporter, backend, frontend, swagger-ui, prometheus и grafana. Это многокомпонентная система с одним прикладным backend.

РАЗДЕЛ 03

Модель хранения данных

Содержимое и метаданные разделены. MongoDB позволяет искать файлы и проверять владельца, не читая объект целиком. S3-интерфейс используется для передачи байтов и не зависит от отображаемого имени файла.

Коллекция	Основные поля и назначение
users	_id, email, password_hash, quota_bytes, used_bytes, created_at. Учётные записи и персональный баланс места. Хеш пароля исключён из JSON-ответов.
files	_id, owner_id, name, size, content_type, object_key, share_token, created_at. Каталог файлов и параметры публичного доступа.
alerts	fingerprint, name, status, severity, summary, description, labels, starts_at, ends_at, updated_at. Полученные оповещения Grafana.

Связь объекта с пользователем

```
users/<userID>/<fileID>
```

Это ключ объекта внутри бакета **S3_BUCKET**. Например, при переименовании файла меняется только **files.name**; S3-объект сохраняет прежний ключ. Отдельные пользователи могут иметь файлы с одинаковыми именами.

Индексы и выборки

• Уникальный email не позволяет создать повторную учётную запись с тем же нормализованным адресом.

• Индекс owner_id + created_at поддерживает выборку файлов владельца; object_key уникален. Публичный share_token имеет уникальный sparse-индекс.

• Оповещение идентифицируется парой fingerprint + starts_at. Переход в resolved обновляет запись того же инцидента.

Каталог возвращает не более **1000 файлов**, от новых к старым. Поиск использует экранированную подстроку имени. Разбиение списка на страницы не предусмотрено.

mongo-data и s3-data сохраняются при обычном пересоздании контейнеров. Для восстановления сервиса нужны согласованные копии обеих частей: одной базы метаданных недостаточно.

РАЗДЕЛ 04

Авторизация и доступ

Регистрация

Клиент отправляет email и пароль на **POST /api/auth/register**. Backend удаляет пробелы по краям email, переводит его в нижний регистр и проверяет формат. Длина пароля ограничена диапазоном **8-72 байта**, поскольку код проверяет длину Go-строки.

Пароль хешируется bcrypt. В MongoDB записывается пользователь с квотой из USER_QUOTA_BYTES. Успех возвращает HTTP 201, данные пользователя, JWT и время истечения. Повторный email приводит к HTTP 409.

Вход и проверка запросов

При входе backend находит пользователя по email и сравнивает пароль с bcrypt-хешем. Неверные данные дают HTTP 401. JWT подписывается алгоритмом **HS256**; в sub хранится ID пользователя, в exp - срок действия. По умолчанию срок составляет **24 часа**.

```
Authorization: Bearer <JWT>
```

gRPC-interceptor проверяет подпись, разрешённый алгоритм, обязательный exp и корректность ID. Для операций с личными файлами MongoDB-запрос содержит одновременно ID файла и ID владельца. Чужой или отсутствующий файл не выдаётся пользователю.

Поведение браузера

JWT сохраняется в localStorage. При обычных API-запросах JavaScript добавляет заголовок Authorization. Для открытия и скачивания обычной ссылкой используется параметр **access_token** в URL. При HTTP 401 клиент удаляет локальную сессию и предлагает войти снова.

Три разных механизма доступа

Механизм	Где используется
JWT пользователя	Личные файлы, профиль, просмотр оповещений.
share_token	Публичное скачивание конкретного файла без JWT.
Служебные токены	ALERT_WEBHOOK_TOKEN для Grafana; S3_EVENT_TOKEN для событий хранилища.

Выход удаляет JWT из браузера. Серверного отзыва токена и refresh-токенов нет. Публичная ссылка действует до её отключения или удаления файла.

РАЗДЕЛ 05

Как загружается файл

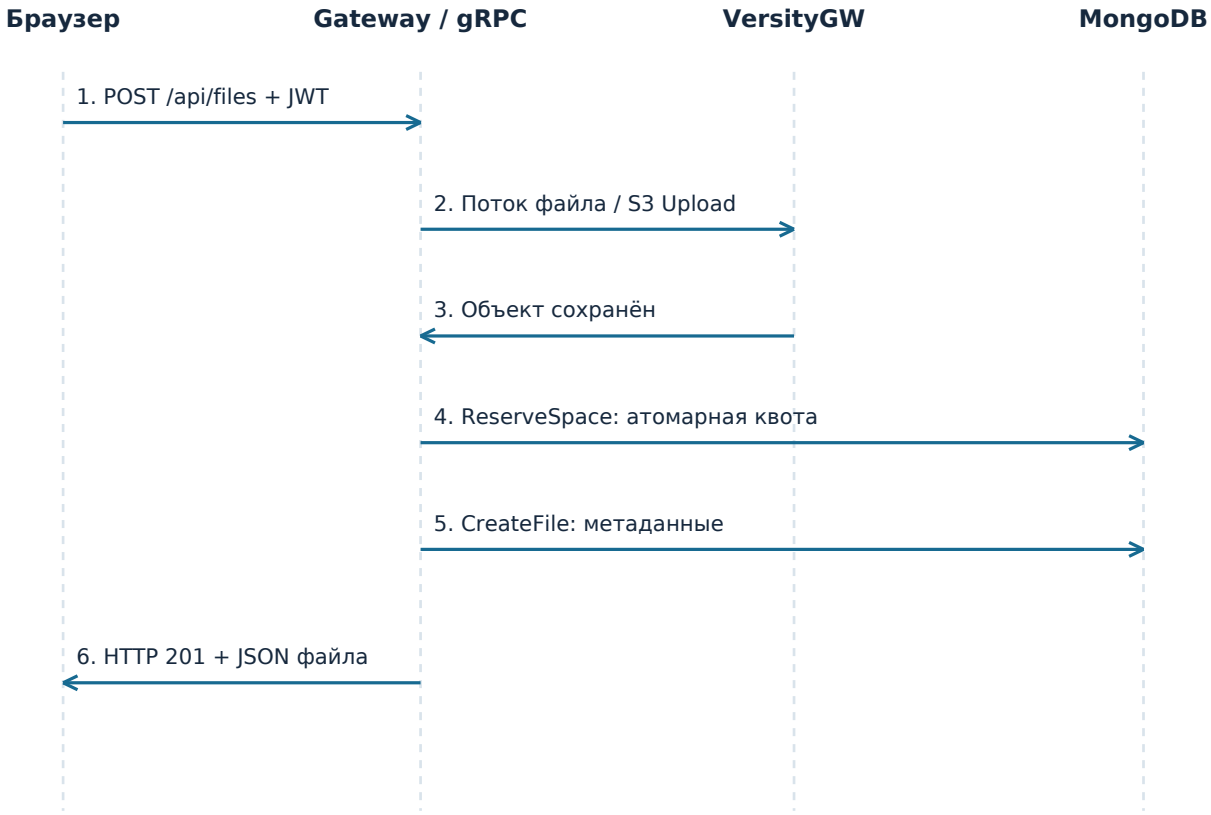


Рисунок 2. Упрощённая последовательность успешной загрузки.

1. Браузер отправляет multipart/form-data с полем **file**. Прогресс измеряется через XMLHttpRequest. nginx передаёт тело без предварительной буферизации на диск.

2. Backend читает текущий баланс и вычисляет предел: **min(остаток квоты, MAX_UPLOAD_BYTES)**. Имя очищается от компонентов пути; тип содержимого определяется по расширению. Генерируются ID и ключ S3-объекта.

3. HTTP-адаптер отправляет gRPC-чанки до 64 КиБ; сервис передаёт reader в S3 uploader. AWS SDK настроен на части по **16 МиБ** и параллелизм **3**. Весь файл не загружается в память целиком, но память нужна для рабочих буферов частей.

4. После записи объекта точный размер резервируется в MongoDB атомарным условным обновлением: used_bytes + size ≤ quota_bytes. Это повторно проверяет квоту при конкурирующих загрузках. Затем создаётся запись files и возвращается HTTP 201.

Обработка ошибок: превышение лимита даёт HTTP 413. Если не удалось зарезервировать место, объект удаляется. Если не удалось записать метаданные, backend освобождает квоту и пытается удалить объект. Это компенсирующие операции, а не общая транзакция MongoDB и S3.

РАЗДЕЛ 06

Жизненный цикл файла

Скачивание и предпросмотр

Backend проверяет владельца, получает метаданные и открывает объект через GetObject. Бизнес-сервис возвращает поток байтов; handler преобразует его в HttpBody, gateway отдаёт бинарный HTTP-ответ. Content-Type и Content-Length берутся из метаданных; Content-Disposition задаёт скачивание либо **inline** при параметре **inline=1**.

Интерфейс предлагает открытие для image/*, video/*, audio/* и text/*. Реальная возможность просмотра зависит от браузера и формата. Ответ содержит X-Content-Type-Options: nosniff и Content-Security-Policy: sandbox. Поддержка Range-запросов в обработчике не реализована.

Переименование и общий доступ

PATCH меняет отображаемое имя в MongoDB. При публикации создаётся случайный токен из 16 байтов, представленный 32 шестнадцатеричными символами. URL вида **/api/public/{token}** выдаёт файл любому обладателю ссылки. DELETE для share удаляет токен из метаданных.

Удаление из приложения

Сначала удаляется объект S3, затем запись files и освобождается место в users.used_bytes. Метод FindOneAndDelete возвращает удалённую запись только одному обработчику. Благодаря этому обычное удаление и пришедшее одновременно событие S3 не освобождают квоту дважды в штатном сценарии.

Удаление напрямую из VersityGW

Шлюз вызывает **POST /internal/s3-events?token=...**. Backend проверяет служебный токен, имя бакета и тип s3:ObjectRemoved:*, находит файл по object_key, удаляет метаданные и возвращает место владельцу. nginx не проксирует этот служебный маршрут в backend.

Если событие пропущено, например во время остановки backend, отсутствие объекта обнаруживается при следующем скачивании: метаданные очищаются, клиент получает HTTP 404. Одна только загрузка списка файлов не выполняет такую сверку.

Загрузка объектов напрямую через S3 или интерфейс VersityGW не создаёт файл в приложении: не формируются owner_id и остальные метаданные. Автоматического импорта произвольных объектов нет.

РАЗДЕЛ 07

Мониторинг системы

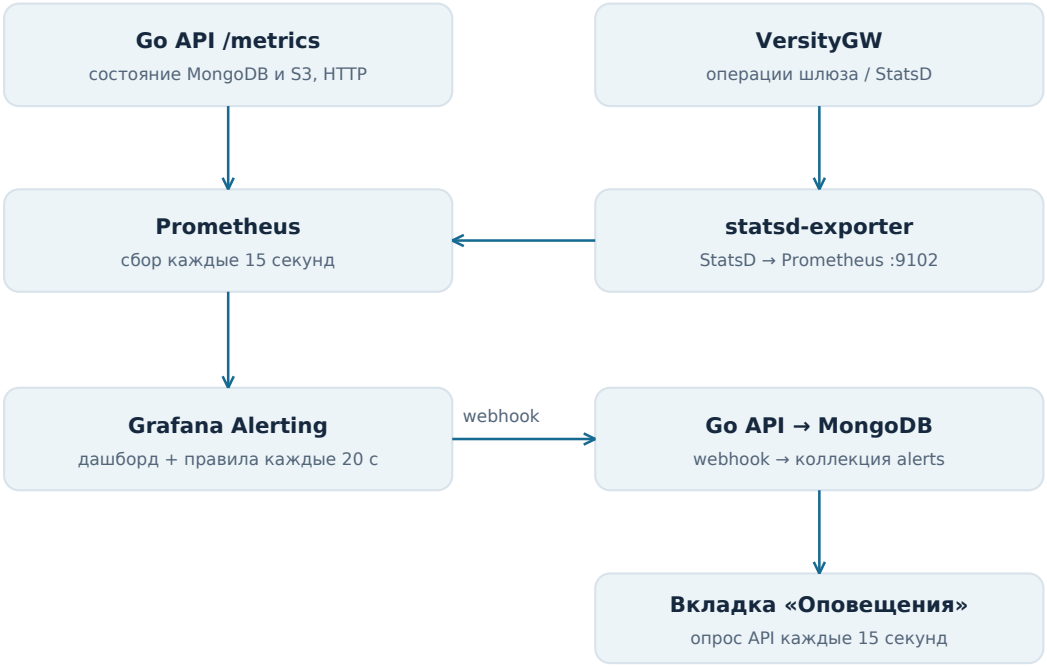


Рисунок 3. Сбор метрик и путь инфраструктурного оповещения.

Backend публикует метрики по **/metrics**. Prometheus опрашивает backend, statsd-exporter и самого себя каждые **15 секунд**. VersityGW отправляет свои показатели по StatsD; exporter преобразует их в формат Prometheus.

Фоновый монитор backend по умолчанию каждые **10 секунд** проверяет MongoDB и S3-бакет, измеряет задержку и обновляет показатели числа пользователей, файлов, занятого места и суммарных квот.

Группа	Примеры метрик
HTTP	cloud_http_requests_total; cloud_http_request_duration_seconds
Файлы и хранилище	cloud_storage_operations_total; cloud_storage_uploaded_bytes_total; cloud_storage_downloaded_bytes_total
Состояние и учёт	cloud_dependency_up; cloud_users_total; cloud_files_total; cloud_storage_used_bytes
События	cloud_auth_events_total; cloud_storage_events_total; cloud_storage_sync_removed_files_total; cloud_alerts_received_total

Grafana получает источник Prometheus, дашборд и правила автоматически из provisioning-файлов. Дашборд показывает доступность компонентов, RPS, статусы HTTP, p50/p95/p99, трафик, квоты, входы и синхронизацию.

РАЗДЕЛ 08

Правила и доставка алертов

Правило	Условие	Выдержка for
BackendDown	up{job="backend"} < 1; отсутствие данных также считается проблемой.	30 с
ObjectStorageDown	Проверка S3-бакета не проходит.	30 с
MongoDBDown	Проверка ping MongoDB не проходит.	30 с
HighErrorRate	Доля ответов 5xx выше 5% по окну 5 минут.	2 мин
HighApiLatency	p95 выше 1 с по окну 5 минут для auth/*, me и alerts.	5 мин
StorageUsageHigh	Отношение занятого места к сумме квот выше 80%.	5 мин
StorageOperationErrors	Более 5 ошибок put/get/delete за 5 минут.	1 мин
LoginBruteforce	Более 20 неудачных входов за 5 минут.	0 с

Grafana вычисляет группу правил каждые **20 секунд**. Параметр for задаёт, сколько условие должно сохраняться до срабатывания. К этому добавляются интервалы проверок, сбора метрик и доставки; алерт не обязан появляться мгновенно.

Доставка и отображение

Контактная точка отправляет POST на **/api/alerts/webhook** с Bearer-токеном ALERT_WEBHOOK_TOKEN. По умолчанию уведомления группируются по папке и имени; group_wait = 10 с, group_interval = 30 с, repeat_interval = 1 ч.

Backend сохраняет firing и последующий resolved через upsert по fingerprint + starts_at. Интерфейс запрашивает последние **100 оповещений** раз в 15 секунд и показывает число активных. Список общий для всех авторизованных пользователей; отдельной роли администратора нет.

Если MongoDB недоступна, backend пишет уведомление в журнал и возвращает HTTP 503 для повторной доставки Grafana. Если недоступен сам backend, вкладка приложения также недоступна: для наблюдения нужен отдельно доступный интерфейс Grafana. Telegram, email и Slack в текущей конфигурации не подключены.

РАЗДЕЛ 09

Развёртывание и конфигурация

Для запуска нужны Docker Engine, плагин Docker Compose и доступ к реестрам образов. Команды выполняются из корня проекта. Перед запуском в .env задаются собственные секреты; значения секретов в этот документ не включены.

```
cp .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs -f backend
```

Адрес на Docker-хосте	Назначение
http://localhost:8080	Пользовательский веб-интерфейс; /api/* через nginx.
http://localhost:8080/swagger/	Интерактивная документация OpenAPI.
http://localhost:7070	S3-совместимый API VersityGW.
http://localhost:7071	Служебный веб-интерфейс VersityGW.
http://localhost:9090	Prometheus.
http://localhost:3000	Grafana. Учётные данные задаются GRAFANA_ADMIN_USER / PASSWORD.

Основные параметры

Переменные	Назначение / значение по умолчанию
USER_QUOTA_BYTES	1 073 741 824 байта = 1 ГиБ на нового пользователя.
MAX_UPLOAD_BYTES	536 870 912 байт = 512 МиБ на один файл.
MONGO_USER / MONGO_PASSWORD	Доступ backend к MongoDB внутри сети Compose.
S3_ACCESS_KEY / S3_SECRET_KEY / S3_BUCKET	Ключи шлюза и имя бакета (cloud-files).
JWT_SECRET / ALERT_WEBHOOK_TOKEN / S3_EVENT_TOKEN	Подпись JWT и защита двух видов служебных уведомлений.

JWT_TTL, S3_REGION и DEPENDENCY_CHECK_INTERVAL поддерживаются Go-конфигурацией, но не пробрасываются текущим compose-файлом. Чтобы изменить их при запуске в Compose, нужно добавить соответствующие переменные в environment backend. Изменение USER_QUOTA_BYTES не обновляет квоты уже созданных пользователей.

РАЗДЕЛ 10

Интерфейсы REST API

Авторизованные методы используют Bearer JWT. Тело обычных запросов и ответов - JSON; загрузка файлов использует multipart/form-data. Успешное удаление и переименование возвращают HTTP 204 без тела.

Метод и путь	Действие / доступ
POST /api/auth/register	Создать пользователя; без JWT.
POST /api/auth/login	Получить JWT; без JWT.
GET /api/me	Профиль, квота и занятое место; JWT.
GET /api/files?q=...	Каталог / поиск по имени; JWT.
POST /api/files	Загрузка, поле file; JWT.
GET /api/files/{id}/download	Скачивание; inline=1 для просмотра; JWT.
PATCH /api/files/{id}	Переименование, JSON {name}; JWT.
DELETE /api/files/{id}	Удаление объекта и метаданных; JWT.
POST /api/files/{id}/share	Создание / получение публичной ссылки; JWT.
DELETE /api/files/{id}/share	Отзыв публичного доступа; JWT.
GET /api/public/{token}	Публичное скачивание; без JWT.
GET /api/alerts	Общий список последних оповещений; JWT.
POST /api/alerts/webhook	Приём уведомлений; служебный Bearer-токен.
GET /api/openapi.yaml	Встроенная в Go-бинарник спецификация API.

Служебные маршруты внутри сети

GET /healthz - жив ли HTTP-процесс; **GET /readyz** - доступны ли MongoDB и S3, иначе HTTP 503; **GET /metrics** - метрики; **POST /internal/s3-events** - события шлюза. Эти пути не направляются nginx в backend снаружи.

Типовые ошибки: 400 - некорректные данные; 401 - нет доступа; 404 - файл не найден; 409 - повторный email; 413 - превышен размер или квота; 500 - внутренняя ошибка; 503 - недоступна зависимость в проверке готовности либо хранилище оповещений.

РАЗДЕЛ 11

Эксплуатация и демонстрация

Порядок старта компонентов

MongoDB запускается первой и проходит healthcheck. Backend подключается к базе, создаёт индексы и начинает обслуживать HTTP. Затем стартует VersityGW: при запуске шлюз проверяет webhook событий, поэтому ему уже нужен работающий backend. Создание S3-бакета выполняется backend в фоне с повтором каждые 2 секунды.

Healthcheck backend использует **/healthz**, а не /readyz: ожидание готовности S3 здесь создало бы цикл зависимостей. Приложение корректно обрабатывает SIGTERM и даёт активным HTTP-запросам до 15 секунд на завершение.

Контейнеры и постоянные данные

Backend собирается в два этапа: Go 1.26 Alpine компилирует статический бинарник, Alpine 3.22 запускает его от непривилегированного пользователя UID 10001. Frontend использует nginx 1.29 Alpine. Политика перезапуска сервисов - unless-stopped.

Тома mongo-data и s3-data содержат прикладные данные; prometheus-data - временные ряды с хранением 15 дней; grafana-data - состояние Grafana. Обычная команда docker compose down сохраняет именованные тома. Флаг **-v удаляет их и приводит к потере данных стенда**.

Сценарий показа на защите

Шаг	Ожидаемое наблюдение
1. Регистрация двух пользователей	У каждого свой каталог и собственная квота.
2. Загрузка и поиск	Файл появляется в каталоге, занятое место увеличивается.
3. Переименование и скачивание	Новое имя отображается; содержимое сохраняется.
4. Публичная ссылка	Работает в приватном окне; после отзыва доступа возвращает 404.
5. Удаление в VersityGW WebUI	При доставке события исчезает запись, освобождается квота.
6. Краткая остановка VersityGW	В Grafana возникает ObjectStorageDown; после запуска - resolved.

Для шага 6: docker compose stop versitygw, затем docker compose start versitygw. Проводить на учебном стенде; это прерывает операции с файлами. Приведён сценарий проверки, а не отчёт о выполненном испытании.

РАЗДЕЛ 12

Границы текущего решения

Для курсовой важно различать реализованные механизмы и свойства, которые потребовали бы дальнейшей доработки. Ниже перечислены ограничения, видимые в исходном коде и Compose-конфигурации.

Надёжность и согласованность

• Один экземпляр MongoDB и один шлюз с POSIX-томом не обеспечивают кластерную отказоустойчивость. Сбой Docker-хоста влияет на весь стенд.

• Запись объекта, учёт квоты и запись метаданных выполняются отдельными действиями. При аварийном завершении процесса возможны объекты без метаданных или расхождение used_bytes; периодического полного восстановления согласованности нет.

• События удаления, пропущенные во время простоя backend, не воспроизводятся самим приложением. Резервная очистка происходит только при скачивании отсутствующего объекта.

• Атомарное удаление записи предотвращает штатное двойное освобождение квоты, но удаление метаданных и уменьшение used_bytes не объединены в транзакцию.

Безопасность и управление

• TLS в конфигурации nginx не настроен. JWT в URL скачивания может попадать в историю браузера и журналы прокси; хранение в localStorage требует защиты от XSS.

• Есть алерт о неудачных входах, но нет ограничения частоты запросов, блокировки перебора, MFA, восстановления пароля и серверного отзыва JWT.

• Служебные панели и S3-порты опубликованы на хосте. Для внешнего размещения нужны отдельные сетевые ограничения и собственные секреты.

Функциональные границы и развитие

Нет папок, версий объектов, корзины, срока действия публичной ссылки, проверки файлов антивирусом и пагинации каталога. Образы с тегом latest не закрепляют точную версию инфраструктуры. Подсеть 100.100.0.0/24 из override-файла подобрана для локальной среды и может потребовать изменения на другом хосте.

Возможные направления развития: HTTPS и безопасная выдача временных ссылок, резервное копирование, сверка MongoDB и S3 по расписанию, разделение ролей, rate limiting, фиксация образов по digest, кластерное хранение и оркестрация. Эти функции в текущем проекте не заявляются как реализованные.

РАЗДЕЛ 13

Карта реализации и выводы

Документ составлен по локальным исходным файлам. Ссылки ниже служат навигацией по реализации и позволяют обосновать описание на защите.

Файл / каталог	Что подтверждает
cmd/server/main.go	Конфигурация, логгер, сигналы ОС и запуск приложения.
internal/app/app.go; app/grpc; app/http	Сборка зависимостей и lifecycle серверов; настройка gateway.
internal/service/s3; internal/domain	Бизнес-операции, независимые модели и ошибки.
internal/grpc/s3	Конвертация protobuf и доменных моделей; вызовы сервиса.
internal/app/monitor.go; pkg/auth	Readiness, мониторинг зависимостей; bcrypt и JWT.
internal/repo/mongo/mongo.go	Модели, индексы, запросы, атомарное резервирование места и удаление записи.
internal/repo/s3/s3.go; internal/metrics/metrics.go	S3-клиент, параметры uploader и определения метрик.
web/static/app.js; web/nginx.conf	Поведение браузера, прогресс, JWT, проксирование и передача потоков.
docker-compose.yml; deployments/Dockerfile	Сервисы, тома, порты, зависимости запуска и сборка backend.
configs/prometheus/; configs/grafana/	Сбор метрик, дашборды, правила и доставка алертов.
proto/cloud/v1; gen/go; scripts	Контракт, сгенерированные RPC/gateway и Swagger 2.0.

Вывод для курсовой работы

Cloud Storage реализует полный базовый цикл работы с файлами: от регистрации и передачи содержимого до публикации ссылок, удаления и наблюдения за состоянием инфраструктуры. Разделение метаданных и объектов связывает прикладную логику с S3-совместимым хранением, а контейнеризация делает состав стенда воспроизводимым.

Swagger генерируется командой bash scripts/generate-api.sh. Проверка: go build ./... и go vet ./...; новые тесты при миграции не добавлялись.

РАЗДЕЛ 14

gRPC и разделение слоёв

Структура следует подходу проекта auth: сборка приложения, запуск серверов, обработчики и бизнес-операции разделены. Имя s3 используется для бизнес-сервиса и gRPC-обработчиков; адаптеры хранения находятся в repo.

```
Browser → nginx → grpc-gateway → gRPC handlers
↓
service/s3 + domain
↓
repo/mongo · repo/s3
```

Слой	Ответственность
cmd/server	Конфигурация, логгер, сигналы ОС, запуск app.App.
internal/app	Сборка зависимостей и lifecycle. app/grpc и app/http запускают и останавливают серверы.
internal/grpc/s3	Преобразование protobuf ↔ domain, вызов сервиса, преобразование ошибок. Interceptors проверяют авторизацию.
internal/service/s3	Квоты, владелец файла, регистрация, ссылки, компенсации и обработка событий. Нет protobuf, HTTP или BSON.
internal/domain	Независимые модели User, File, Alert, Upload, Download и доменные ошибки.
internal/repo	MongoDB/BSON и S3-адаптеры. Модели хранения преобразуются в доменные.

Каждый RPC принимает свой MethodRequest и возвращает MethodResponse. LoginRequest содержит Credentials, LoginResponse - Session; потоковые ответы содержат HttpBody. HTTP-аннотации сохраняют прежний JSON и исходные байты файлов.

Генерация HTTP и Swagger

Аннотации google.api.http связывают RPC с прежними HTTP-путями. Скачивание генерируется как поток google.api.HttpBody. Единственный ручной прикладной HTTP-адаптер разбирает multipart-загрузку и отправляет gRPC-чанки до 64 КиБ.

Контракт: proto/cloud/v1/storage.proto. Код: gen/go/cloud/v1. Скрипт scripts/generate-api.sh генерирует также Swagger 2.0 и добавляет описание multipart из api/http.yaml. JSON int64 передаётся десятичной строкой; frontend учитывает это.

Проверены прямой gRPC-клиент и HTTP gateway: авторизация, пустой файл и бинарный файл 6 МиБ, скачивание без искажения байтов, ссылки, удаление, квоты и native webhook. Проверочный стенд изолирован от VPS и пользовательских данных.